

Algorithm Analysis

Graph Coloring

Felix Pîrlog, Elena-Bianca Geică, and Nurjahan Nepesova

Faculty of Automatic Control and Computer Science
National University of Science and Technology POLITEHNICA Bucharest

Abstract. This project addresses the graph coloring problem (NP-Hard), comparing the efficiency of Greedy, Welsh-Powell, and DSatur algorithms (implemented in Standard and Optimized variants). The objective was to find the best balance between execution speed and the minimum number of colors obtained, running the solutions on a benchmark set of 78 test cases. Additionally, we used Valgrind to identify code bottlenecks and optimize memory consumption.

The results clearly highlight the trade-offs: **Welsh-Powell** is the fastest, but yields a lower-quality solution. Conversely, **Optimized DSatur** proves to be the best overall: it achieved a more efficient coloring in a very short time, being, on average, 4 times faster than its initial variant.

Keywords: Graph Coloring · Greedy Heuristics · NP-Hard
· Exhaustive Search · Backtracking · Greedy · DSatur · Welsh-Powell

1 Introduction

1.1 Problem Description

Graph coloring is a fundamental problem in graph theory, with numerous practical applications across various domains such as scheduling, resource allocation, communication networks, and social analysis [11]. In general, the problem consists of assigning a minimum number of colors to the vertices of a graph such that adjacent vertices receive different colors, allowing for the elegant modeling of conflicts or incompatibilities in real-world systems [9].

From a computational perspective, graph coloring presents varying levels of difficulty. The decision problem of determining whether a graph can be colored using a fixed number of colors is NP-complete, as a proposed solution can be verified quickly, but finding one is difficult. In contrast, determining the chromatic number $\chi(G)$ (the absolute minimum number of colors required) is an NP-hard problem, meaning there is no known algorithm that can solve it in polynomial time for any arbitrary instance [9].

This theoretical barrier has driven the development of a wide range of approaches, spanning from exact algorithms based on backtracking techniques to heuristic and meta-heuristic methods that yield near-optimal solutions in reasonable time [7].

In this project, we aim to explore the **Greedy**, **Welsh-Powell**, and **DSatur** heuristics by implementing these algorithms in the C programming language and analyzing their efficiency and solution quality, correlating how the structure of the graph influences the performance of the coloring process.

1.2 Examples of Practical Applications

Team Building

Graph coloring can be used to divide a group of people into teams such that incompatible individuals are not placed in the same team. In this model, each person is represented by a vertex in the graph, and an edge between two vertices indicates an incompatibility relationship (for instance, friends who should not be assigned to the same team). A valid graph coloring corresponds to a partition into groups, where the number of colors used represents the minimum number of teams required to satisfy all constraints [9].

Register Allocation in Compilers

In compilers, graph coloring is used to allocate program variables to processor registers. Each variable is represented as a vertex, and an edge between two vertices indicates that the variables are active simultaneously and cannot be stored in the same register. The resulting graph is called an interference graph, and its coloring corresponds to a register allocation. The objective is to use as few registers as possible, leading to more efficient program execution [1], [2].

Class Scheduling

Another application is course scheduling in educational institutions. Courses are represented as vertices of a graph, and an edge between two vertices indicates shared students or the same instructor, signaling that those courses cannot be scheduled simultaneously. Assigning colors corresponds to placing courses into time slots. Thus, a valid coloring guarantees that all courses scheduled in the same time slot are compatible, optimizing the utilization of classrooms and available resources [3], [8].

Printed Circuit Board Testing

An engineering application lies in the efficient detection of short circuits on printed circuit boards (PCBs). Each electrical network on the board is represented by a vertex, and an edge between two vertices indicates that the tracks are physically close to each other, creating a risk of a short circuit. A valid graph coloring corresponds to dividing the networks into independent test groups. As a result, all tracks sharing the same color can be tested simultaneously, and the chromatic number represents the minimum number of test steps required for the entire board [5], [4].

2 NP-Hard Proof

To establish that the *3-Coloring* problem belongs to the NP-Hard class, we will demonstrate the polynomial-time reducibility of the *3-SAT* problem to *3-Coloring* ($3\text{-SAT} \leq_p 3\text{-Coloring}$). Since 3-SAT is an established NP-Complete problem, the possibility of reducing it to our problem implies that 3-Coloring is at least as hard.

2.1 Step 1: Problem Definitions

Problem A: 3-SAT

- **Input:** A Boolean expression in Conjunctive Normal Form. It is composed of m clauses $(C_1 \wedge C_2 \wedge \dots \wedge C_m)$, where each clause C_j is a disjunction of exactly 3 literals.

Definitions:

- * A **variable** (x_i) is the atomic element that can receive a truth value (*True* or *False*).
- * A **literal** is a variable (x_i) or its negation $(\neg x_i)$.
- * A **clause** has the form $(l_1 \vee l_2 \vee l_3)$, where l_1 , l_2 , and l_3 are literals.

- * **Example:** A valid input instance could be:

$$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

This expression is satisfiable if we can assign truth values to the variables x_1, x_2, x_3 such that both parentheses evaluate to *True*.

- **Requirement:** Determine whether there exists a truth assignment (True/False) for the expression's variables such that it evaluates to True.

Problem B: 3-Coloring

- **Input:** An undirected graph $G = (V, E)$, where V is the set of vertices and E is the set of edges.
- **Requirement:** Determine whether there exists a valid coloring function $C : V \rightarrow \{c_1, c_2, c_3\}$ using at most 3 colors.

2.2 Step 2: Instance Construction (Transformation Function)

Given an instance of 3-SAT (a Boolean expression), we construct a graph $G = (V, E)$ in polynomial time such that G admits a valid 3-coloring if and only if the expression is satisfiable. The construction is based on three structural components:

1. Control Nodes

Three special interconnected nodes are introduced (forming a clique K_3), serving to fix the meaning of the colors:

- V_{True} (the logical value for True)
- V_{False} (the logical value for False)
- V_{Base} (the logical value for the Base / Neutral node)

In a valid 3-coloring, these nodes must strictly have distinct colors.

2. Variable Component For each variable in the formula, two nodes are generated: one for the positive literal (v_x) and one for the negated literal ($v_{\neg x}$).

- The edge $(v_x, v_{\neg x})$ is added to ensure correctness (the two nodes must have different colors).
- Both nodes are connected to V_{Base} .

Effect: Being adjacent to V_{Base} , variable nodes cannot take the "Neutral" color. They are forced to adopt either the "True" or "False" color.

3. Clause Component For each clause in the expression, a subgraph is constructed that connects the literal nodes to the reference nodes.

Fundamental Property: This component allows a valid 3-coloring **if and only if** at least one literal in the clause has the "True" color. In a situation where all literals are colored with "False", the subgraph cannot be validly colored with the set of 3 colors.

Complexity The number of generated nodes and edges is directly proportional to the number of variables and clauses. Therefore, the construction is performed in polynomial time: $O(n + m)$ (where n represents the number of variables, and m the number of clauses).

2.3 Step 3: Proof of Equivalence

The proof of correctness of the reduction must be valid in both directions.

Direction 1: 3-SAT $\implies$ 3-Coloring

Hypothesis: Suppose that the initial Boolean expression is satisfiable (there exists an assignment that makes all clauses true). We will construct a valid 3-coloring of graph G .

Proof:

1. **Reference nodes** receive fixed colors $(c_{True}, c_{False}, c_{Base})$.

2. **Variables:** For each variable, if its logical value is True, we color the corresponding node with c_{True} and its negation with c_{False} (and vice versa). This coloring is valid according to the hypothesis.
3. **Clauses:** Since the expression is satisfiable, each clause contains at least one literal with the value True (colored with c_{True}). According to the fundamental property of the clause component, the coloring can be validly extended to all internal nodes of the subgraph.

Conclusion: Graph G admits a valid 3-coloring.

Direction 2: 3-Coloring $\implies$ 3-SAT Suppose that the generated graph G admits a valid 3-coloring. We will extract a solution for the Boolean expression:

1. From the construction of the variable subgraph, any variable-type node has either color c_{True} or c_{False} (being connected to V_{Base}).
2. We interpret the colors as logical values: $x_i = \text{True}$ if node v_{x_i} has color c_{True} .
3. We analyze the clause subgraphs. We know that a component of this type does not allow a valid coloring if all input literals have color c_{False} .
4. Since the graph is validly colored, we deduce that in each clause there is at least one literal colored with c_{True} (meaning it has the value True).

Conclusion: All clauses are satisfied simultaneously, so the Boolean expression is satisfiable.

2.4 Step 4: Final Conclusion

Since we have demonstrated that 3-Coloring belongs to the NP class (Nondeterministic Polynomial time - the solution can be verified in polynomial time) and showed the reduction $3\text{-SAT} \leq_p 3\text{-Coloring}$, it follows that the 3-Coloring problem is NP-Complete, hence implicitly the **3-Coloring problem is NP-Hard**.

3 Algorithm Presentation

3.1 How the Algorithms Work

Welsh-Powell Algorithm

It is an efficient graph coloring method used to use as few colors as possible so that two neighboring nodes never share the same color. The core idea is prioritizing "difficult" nodes, i.e., nodes with the most connections (high degree). The algorithm works in three simple steps:

1. **Ordering:** The degree $d(v)$ is calculated for each node, and all nodes are sorted into a list in descending order of their degrees.
2. **Coloring:** The first available color (C_1) is selected and assigned to the first node in the list (the one with the maximum degree). Then, the rest of the list is traversed, assigning the same color to all nodes that are **not** adjacent to those already colored in this step.

3. **Repetition:** The algorithm proceeds to the next color $(C_2, C_3, \dots)$ and repeats step 2 for the remaining uncolored nodes until the entire graph is completely colored [12], [13], [10].

3.2 Complexity Analysis of the Solutions

Let $G(V, E)$ be a graph where V is the number of vertices and E is the number of edges.

- **Greedy:** The algorithm traverses the nodes sequentially once. Its complexity is **linear**, $\mathcal{O}(V + E)$, determined by the need to check the neighbors of each current node. This explains the high performance observed during testing, as the execution time remains negligible even for large graphs.
- **Welsh-Powell:** This algorithm introduces a preliminary step of sorting nodes by degree (descending), also being referred to as Degree-based Greedy.

$$\mathcal{O}(V \log V + E)$$

Although theoretically slower than Greedy, in practice the difference is minimal, as the sorting cost is quickly amortized.

- **DSatur (Degree of Saturation):** The standard version of the algorithm requires searching for the uncolored node with the highest saturation degree at each step. This search traverses all remaining nodes, leading to a **quadratic** complexity:

$$\mathcal{O}(V^2)$$

- **Optimized DSatur:** By using data structures (singly-linked lists, adjacency lists) for fast extraction of the node with maximum saturation, the complexity can be reduced to:

$$\mathcal{O}((V + E) \log V)$$

However, even with this optimization, the algorithm remains significantly slower than static approaches (Greedy, Welsh-Powell) due to the constant overhead of updating dynamic data structures at each coloring step.

Table 1. Complexity Summary

Algorithm	Theoretical Complexity	Growth Type
Greedy	$\mathcal{O}(V + E)$	Linear
Welsh-Powell	$\mathcal{O}(V \log V + E)$	Linear-Logarithmic
DSatur (Standard)	$\mathcal{O}(V^2)$	Quadratic
DSatur (Optimized)	$\mathcal{O}((V + E) \log V)$	Linear-Logarithmic

3.3 Main Advantages and Disadvantages

Welsh-Powell Algorithm

A 2025 study showed that the Welsh-Powell algorithm is a more efficient solution than DSatur for coloring sparse graphs. In this research, the authors implemented and tested the two algorithms on a benchmark set of 46 graphs with sizes ranging between 800 and 9,800 nodes and a low density of 3%. Experimental results highlighted that Welsh-Powell achieved a better chromatic solution in 50% of cases, compared to only 19.6% for DSatur. Additionally, it was concluded that Welsh-Powell is clearly superior in terms of execution speed, being much faster as the graph size increases [6].

We presented the main advantages and disadvantages in the table below:

Advantages	Disadvantages
– Easy to implement: simple and intuitive logic – Speed: much faster than exact algorithms (backtracking) – Efficiency: usually uses fewer colors than a simple random coloring due to initial sorting – Practical: works well on real-world problems such as timetabling	– Not optimal: does not guarantee finding the absolute minimum number of colors – Sorting cost: requires time to sort nodes by degree – Limited: on sparse/special graphs, it may use more colors than necessary – Static: if the graph changes (a node is added), the entire process must be restarted from scratch

Table 2. Analysis of the Welsh-Powell Algorithm

Degree Of Saturation Algorithm

In contrast to Welsh-Powell's static approach, the DSatur (Degree of Saturation) algorithm, although originally proposed in the 1970s, remains one of the most robust sequential heuristics for medium- and high-density graphs [9]. While the aforementioned studies indicate a superior performance for the Welsh-Powell algorithm strictly on sparse graphs, specialized literature confirms that DSatur provides a much more accurate approximation of the optimal chromatic number ($\chi(G)$) in the vast majority of general cases. This qualitative superiority stems from its dynamic nature: by recalculating priorities at each step, the algorithm avoids early "greedy" decisions that could deadlock the graph in later stages, though sacrificing execution speed for accuracy.

We summarized the main advantages and disadvantages of this approach in the table below:

Advantages	Disadvantages
– Superior quality: consistently achieves a lower number of colors, very close to optimal – Adaptability: the dynamic heuristic detects critical areas (cliques) as it colors – Stability: the result is much less dependent on the initial order of the nodes – Ideal tradeoff: provides the best quality/time ratio before resorting to exponential algorithms	– Reduced speed: significantly slower than static algorithms on large graphs – Memory footprint: the optimized variant requires additional space to store saturation degrees – Sensitivity to ties: when multiple nodes have the same saturation degree, arbitrary selection between them can reduce solution quality

Table 3. Analysis of the DSatur Algorithm

4 Evaluation

4.1 Test Set Construction Methodology

We have a total of 78 test cases divided into 4 pillars as follows:

- **Pillar I. Fundamental (Sanity check)** contains tests 1 through 10 and was manually constructed. The number of nodes increases from 5 to 20 with random density. We validate that the algorithms function correctly from a logical perspective.
- **Pillar II. Scalability** contains tests 11 through 49 and was generated using `generator_teste_pilon2_si_3.c`. The number of nodes increases from 100 to 3900 in steps of 100, while the density remains constant at 10%. We measure time efficiency under exponential growth in the number of nodes.
- **Pillar III. Density** contains tests 50 through 69 and was generated using `generator_teste_pilon2_si_3.c`. The number of nodes remains constant at 500, while the density increases from 5% to 100% in steps of 5%. We measure time efficiency under increasing edge density.
- **Pillar IV. Special Graphs** contains tests 70 through 78 and was generated using `generator_teste_pilon4_speciale.c`. It includes complete, complete bipartite, planar, and chordal graphs.

Table 4. Test Set Summary

Pillar	Objective
I Fundamental	Logical correctness
II Scalability	Execution time vs. Size (n)
III Density	Edge density impact
IV Special Graphs	Theoretical accuracy

4.2 Hardware and Software Specifications

Hardware

- CPU: Intel Core i7-13650HX (14 cores, 20 threads) Base Clock 2.60GHz, Boost Clock 4.7GHz
- RAM: 16 GiB DDR5 @ 4800MHz
- Storage: 512 GiB SSD NVMe PCIe Gen4

Software

- OS: Ubuntu 22.04.3 LTS, Linux Kernel 6.6.87.1-microsoft-standard-WSL2
- C Standard: C17

4.3 Testing Methodology and Tools Used

To ensure the accuracy and reproducibility of time measurements, we used the **hyperfine** benchmarking tool. It allowed us to execute the test runs under identical conditions and automatically generate statistical analyses.

The testing workflow was automated via a Bash script that invoked **hyperfine** for each test across all 4 pillars:

```
hyperfine --warmup 2 --min-runs 10 --export-json results.json \
    "../src/algorithm < test.in > /dev/null"
```

Benchmark configuration:

- **--warmup 2**: executed the program twice prior to actual measurements to warm up the CPU cache and bring the operating system to a steady state (eliminating cold-start anomalies).
- **--min-runs 10**: executed each test at least 10 times to compute the mean and standard deviation, filtering out noise caused by background system processes.
- **>/dev/null**: redirected standard output to measure strictly the algorithm's execution time, excluding terminal I/O latency.

4.4 Evaluation Results on the Proposed Test Set

Pillar I

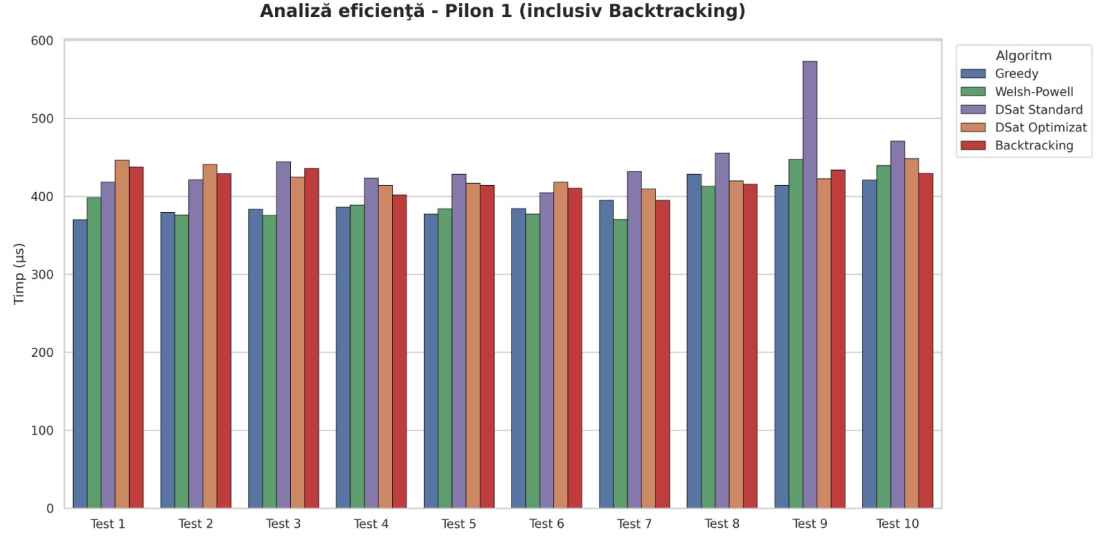


Fig. 1. Efficiency Analysis for Pillar I Tests

On these small graphs, Backtracking does not exhibit exponential execution time overhead, remaining as fast as the heuristic algorithms. Standard DSat occasionally exhibits slightly higher times (e.g., Test 8, Test 9), but the difference remains negligible in this initial benchmark tier.

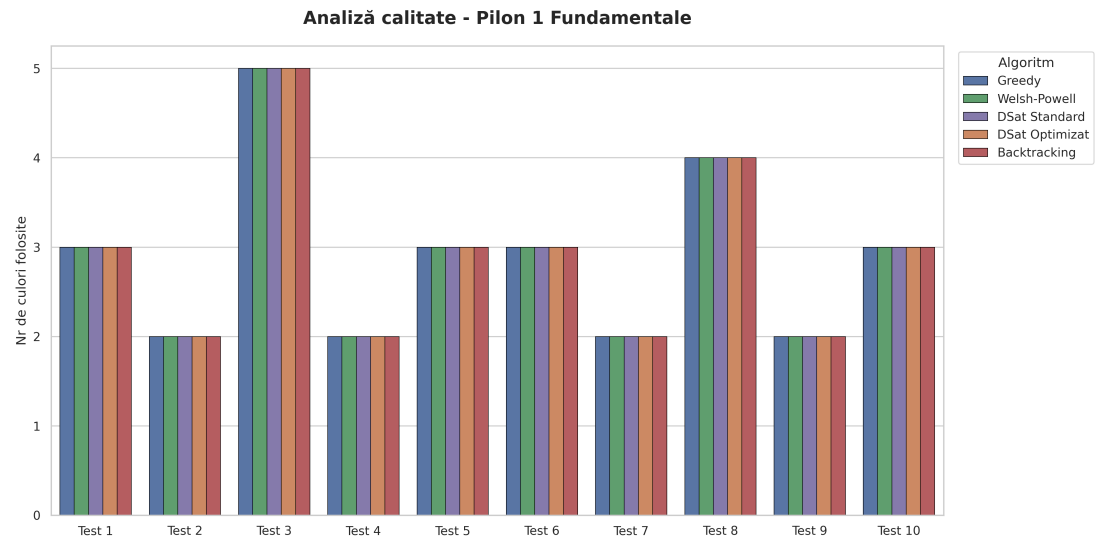


Fig. 2. Quality Analysis for Pillar I Tests.

All 5 algorithms (Greedy, Welsh-Powell, Standard DSat, Optimized DSat, and Backtracking) obtained the exact same chromatic number for every test in Pillar I.

Pillar II

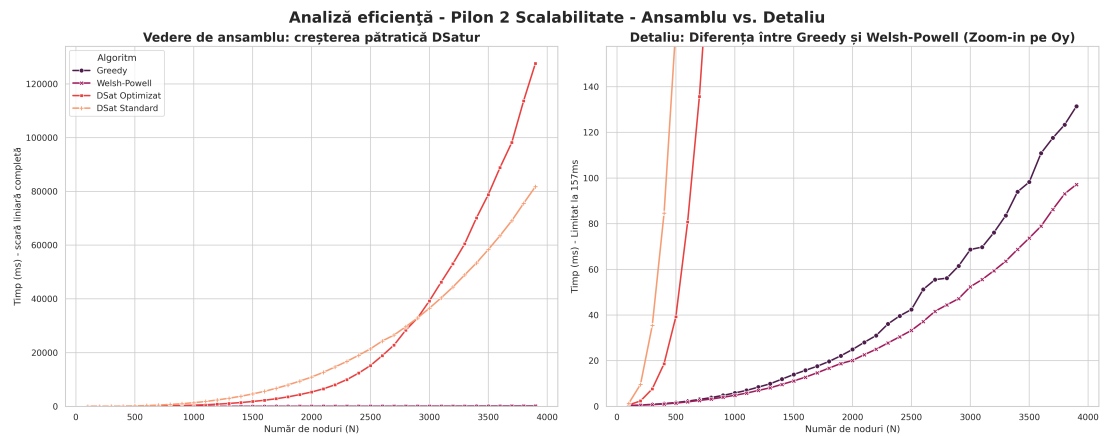


Fig. 3. Efficiency Analysis for Pillar II Tests.

In the overview plot, we observe that the DSatur algorithm family is significantly slower compared to Greedy and Welsh-Powell. The DSatur execution curves follow a steep quadratic trajectory, exceeding 120,000 ms (2 minutes), whereas the other algorithms remain near 0 on this scale.

In the detailed view, we observe the performance comparison among the faster algorithms. Welsh-Powell proves to be the most efficient, maintaining a lower execution time than Greedy, while both remain extremely fast (under 140 ms even at 3900 nodes).

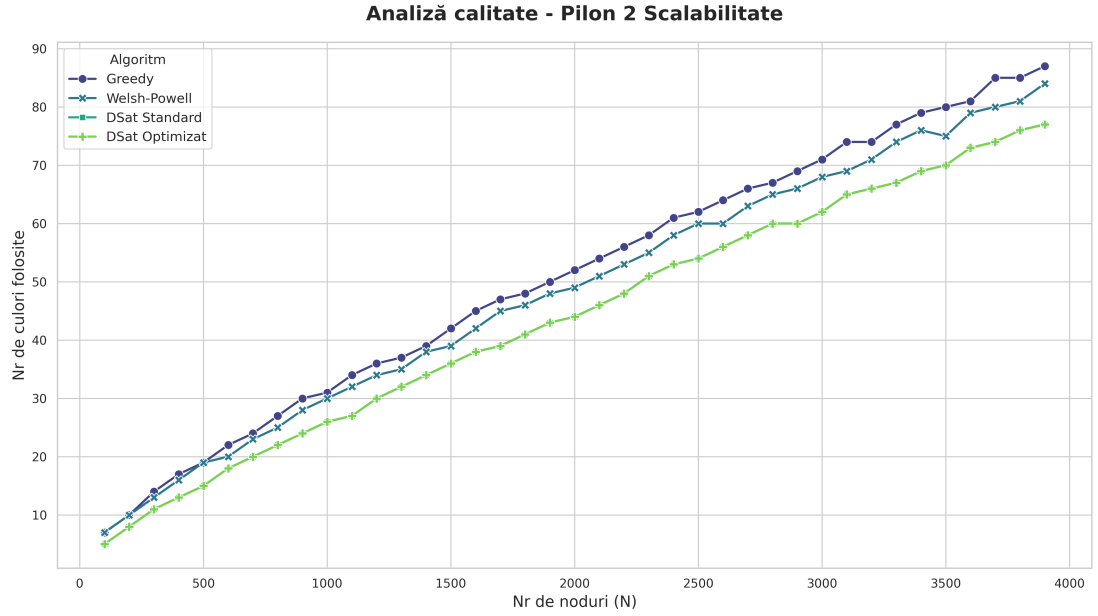
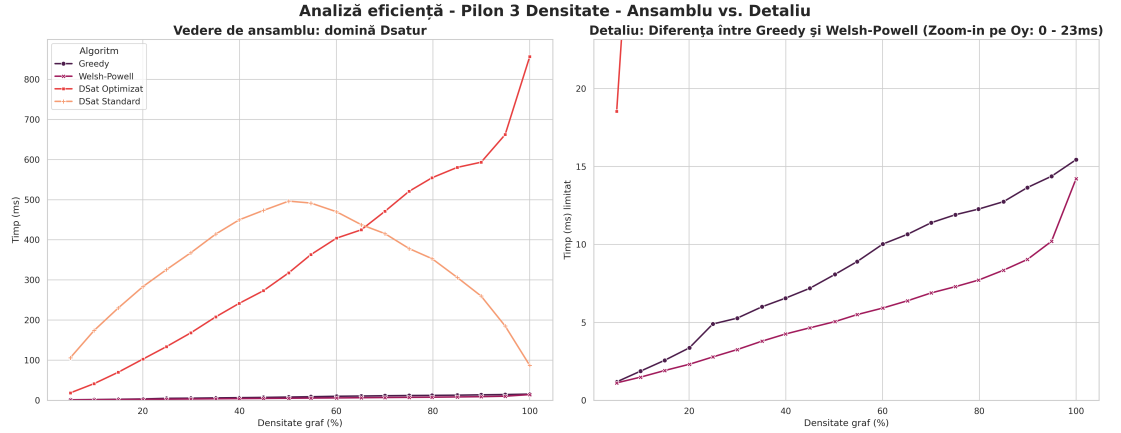


Fig. 4. Quality Analysis for Pillar II Tests.

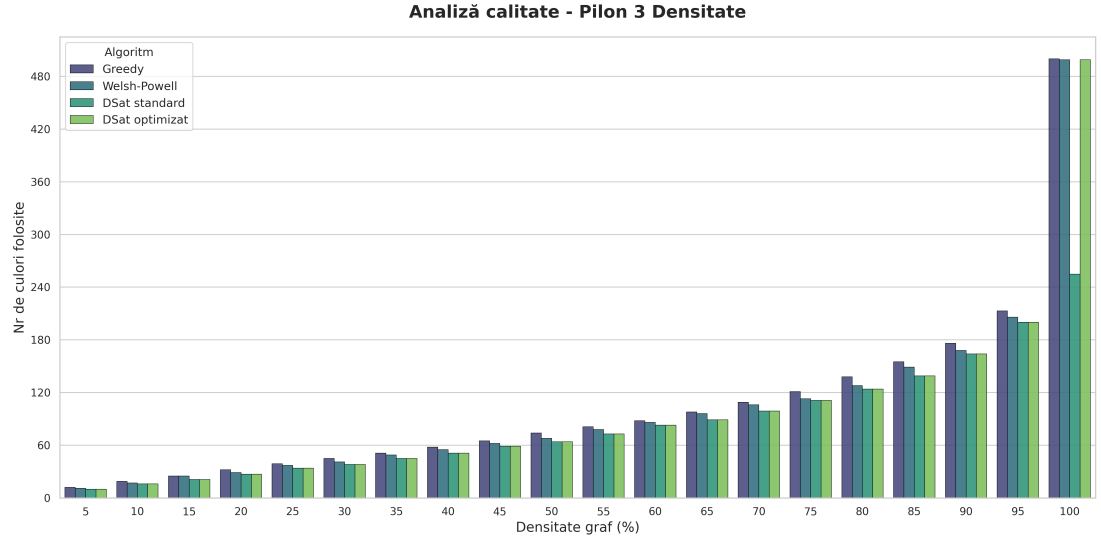
Unlike the behavior observed in Pillar I where all algorithms yielded identical color counts, performance variances emerge here. Greedy is the least effective, consistently requiring the highest number of colors. Standard and Optimized DSat yield the highest solution quality; their curves overlap perfectly, consistently producing the lowest color counts. Welsh-Powell occupies a middle ground, outperforming Greedy while yielding slightly lower solution quality than DSat. The performance gap widens as the vertex count increases. At 3900 nodes, Greedy uses approximately 87 colors, whereas DSat successfully colors the graph using under 78 colors.

Pillar III

**Fig. 5.** Efficiency Analysis for Pillar III Tests.

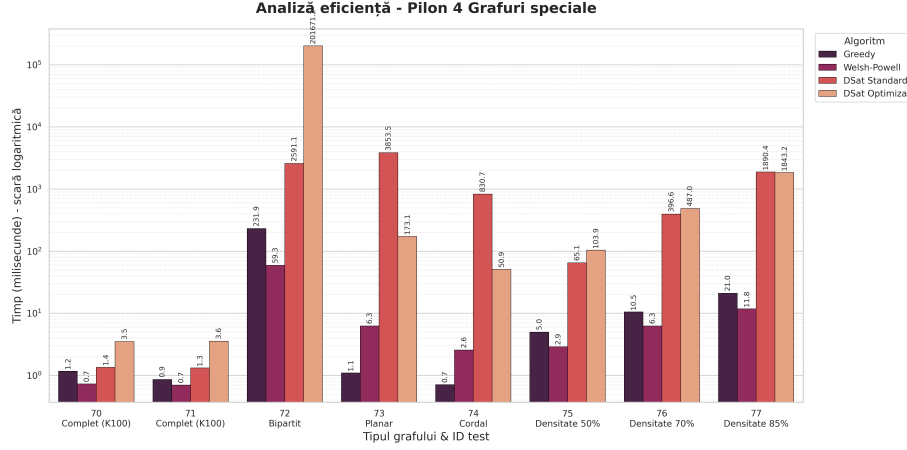
As edge density increases toward 100%:

- Standard DSat exhibits a bell-shaped execution curve—runtime increases up to 60% density, after which it decreases.
- Optimized DSat does not benefit from higher density; its execution time grows continuously and steeply, becoming the slowest algorithm at densities above 70%.
- Welsh-Powell maintains top performance: as seen in the detailed view on the right, Welsh-Powell is consistently faster than Greedy regardless of graph density.
- The performance disparity remains substantial. Fast algorithms finish in under 16 ms, whereas DSatur can exceed 800 ms at maximum density.

**Fig. 6.** Quality Analysis for Pillar III Tests.

Denser graphs consistently require more colors. The chromatic requirement scales steadily from under 20 colors at low density up to nearly 500 at maximum density. DSat algorithms consistently deliver the highest quality (fewest colors used), maintaining a lower bound across the entire graph. The Greedy algorithm consistently yields the lowest quality, systematically using more colors than competing heuristics. At maximum density, solution quality variance diminishes for most algorithms as high connectivity enforces large color requirements, though Standard DSat continues to identify the lowest color count.

Pillar IV

**Fig. 7.** Efficiency Analysis for Pillar IV Tests.

The most prominent anomaly occurs in Test 72 (bipartite graph). Optimized DSat exhibits poor efficiency, spiking above 200,000 ms (2×10^5), while all other algorithms complete almost instantaneously. This highlights that structural optimizations can introduce unintended overhead on specific graph topologies. Conversely, we observed a performance reversal on planar (Test 73) and chordal graphs (Test 74), where Optimized DSat runs significantly faster than Standard DSat. Thus, DSatur efficiency depends heavily on internal graph structure rather than size alone. Regardless of topology, Welsh-Powell maintains consistent minimal runtime, proving to be the most robust algorithm across special graph structures. On complete graphs, all algorithms exhibit identical behavior due to the simple and uniform structure leaving no room for complex heuristic trade-offs.

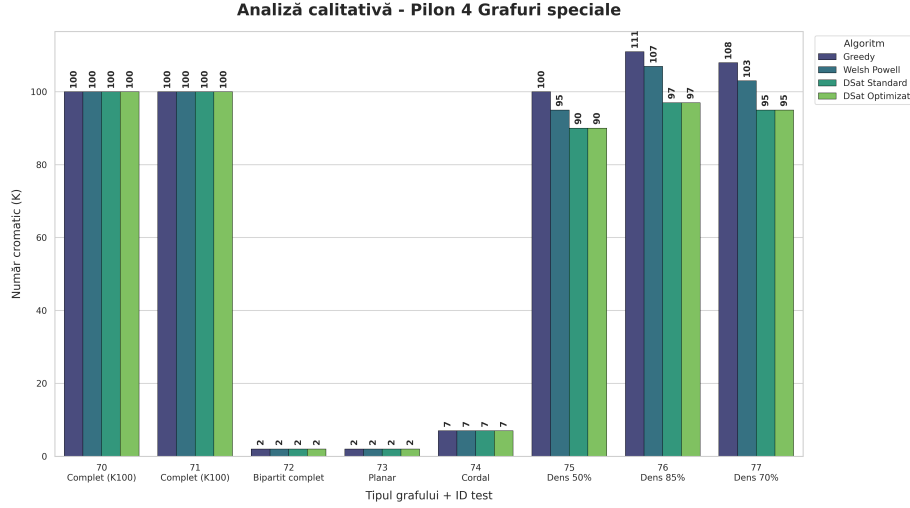


Fig. 8. Quality Analysis for Pillar IV Tests.

For standard mathematical graph families—complete (Tests 70, 71), bipartite (Test 72), planar (Test 73), and chordal (Test 74)—all algorithms performed identically. The rigid structure of these graphs allowed all heuristics to discover the optimal solution. Solution quality variations reappear in tests featuring random edge density (Tests 75, 76, 77). Here, the established hierarchy reemerges: Greedy uses the most colors, DSat variants are the most efficient (saving 10–15 colors compared to Greedy), and Welsh-Powell consistently occupies the middle range, yielding a clear improvement over Greedy without matching DSat’s precision.

4.5 Interpretation of Obtained Values

- 1. Welsh-Powell Superiority:** The Welsh-Powell algorithm proved to be the most efficient in almost all testing scenarios. On planar and chordal graphs, it achieved execution times under 10ms, being several orders of magnitude faster than the DSatur variants. The initial sorting of nodes by degree represents a negligible computational cost compared to the benefits provided during the coloring phase.
- 2. DSatur:** The *DSatur* family of algorithms holds supremacy regarding solution quality (minimum number of colors), which is particularly evident in scalability and density tests. However, this benefit comes at a temporal cost (quadratic growth) and major instability on certain topologies (special graphs, such as bipartite graphs), where the optimized variant performs poorly.
- 3. Irrelevance on Small Graphs:** For small graphs, the choice of algorithm is practically irrelevant. All 5 algorithms, including Backtracking,

yielded qualitatively identical solutions within execution times on the scale of microseconds.

Experimental Analysis and Profiling

The tests were conducted on a dense graph with 364 nodes and 3339 edges. Profiling was performed using Valgrind (memcheck) for memory analysis and Callgrind visualized in KCachegrind for analyzing execution time and call trees.

- * Profiling is the dynamic analysis process of a program to measure precisely what resources it consumes, for instance, how much CPU time is spent in each function and how many RAM bytes are allocated.

Performance Comparison

Table 5 presents the comparative results obtained. The trade-off between execution time and solution quality (the resulting chromatic number) can be observed.

Algorithm	Time (s)	Memory	Colors	Observations
Welsh-Powell	0.012	150.7 KB	9	Fastest, sub-optimal solution
DSatur (Unoptimized)	0.137	24.3 MB	8	High processing overhead
DSatur (Optimized)	0.033	97.1 MB	8	Optimal Time/Quality Ratio
Greedy	0.696	4.1 MB	11	Lowest quality solution
Backtracking	N/A	N/A	N/A	Testing could not be executed on this set

Table 5. Performance Comparison of Implemented Algorithms

Bottleneck Identification

Using profiling tools, we identified the critical functions limiting performance for each algorithm.

1. Welsh-Powell

- **Bottleneck:** Initial sorting (`qsort`).
- **Analysis:** Profiling indicates that 98.44% of the time is consumed in `main`. As the algorithm is static, the coloring traversal is linear and extremely fast. The primary limitation relates to solution quality.

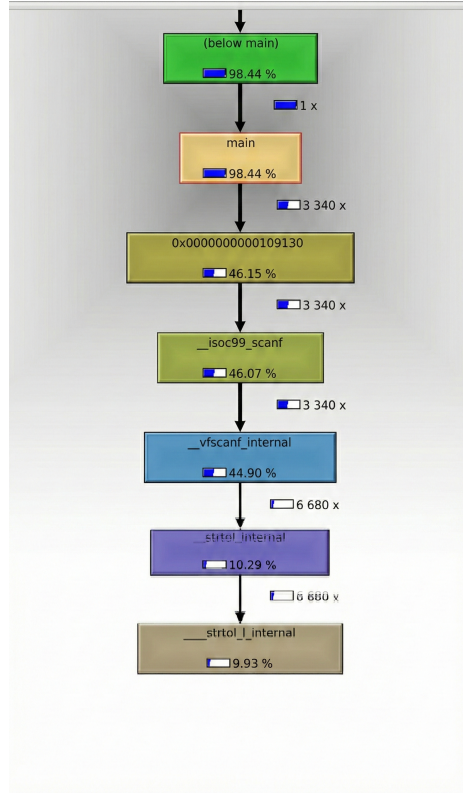


Fig. 9. CallGraph Analysis: Welsh-Powell Algorithm

2. DSatur (Analysis and Optimization)

Initial profiling of the first implementation (unoptimized variant) revealed that the main performance bottleneck was not caused by the heuristic itself, but by repetitive recalculations of saturation degrees.

This observation motivated the decision to implement an optimized variant aimed at eliminating implementation inefficiencies. Thus, the objective was to isolate and highlight the actual *algorithmic bottleneck*, rather than the bottleneck created by inefficient resource management.

- **Unoptimized:** The profiler showed that functions `saturation_of_vertex` (51.19%) and `degree_vertex` (47.15%) saturated the processor. Recalculating the degree at each step involved redundant traversals and a massive number of dynamic allocations (approx. 66,000 `malloc` calls).
- **Optimized:** By modifying data structures and eliminating allocations within the main loop, execution time was reduced from 0.137s to 0.033s. Although memory consumption increased (to store intermediate states), the call tree structure was drastically simplified, demonstrating that the

initial overhead was an implementation defect rather than a fundamental flaw of the algorithm.

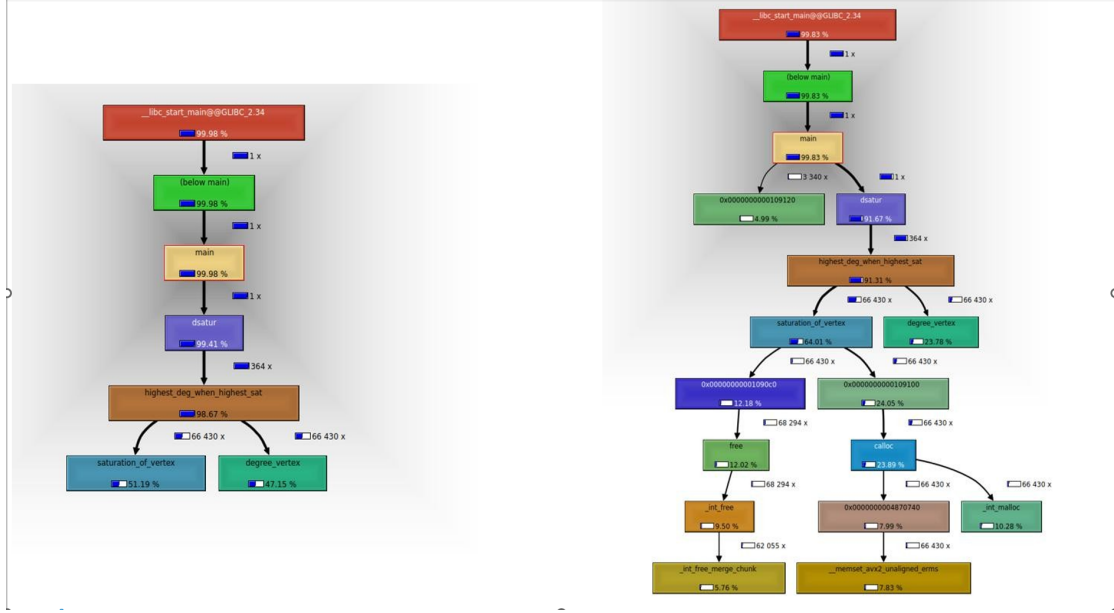


Fig. 10. CallGraph Analysis: Unoptimized DSatur (Left) vs. Optimized (Right). Notice the elimination of repetitive calls.

3. Greedy and Backtracking

- **Greedy:** Callgrind analysis revealed a surprising fact: 78.18% of the time was spent in the `readGraph` function. The coloring algorithm itself is fast, but data input inefficiency affected overall execution time. However, it was observed that the next function with a significant execution time of 12.36% is the one determining the chromatic number, leading to the conclusion that the bottleneck resides there.

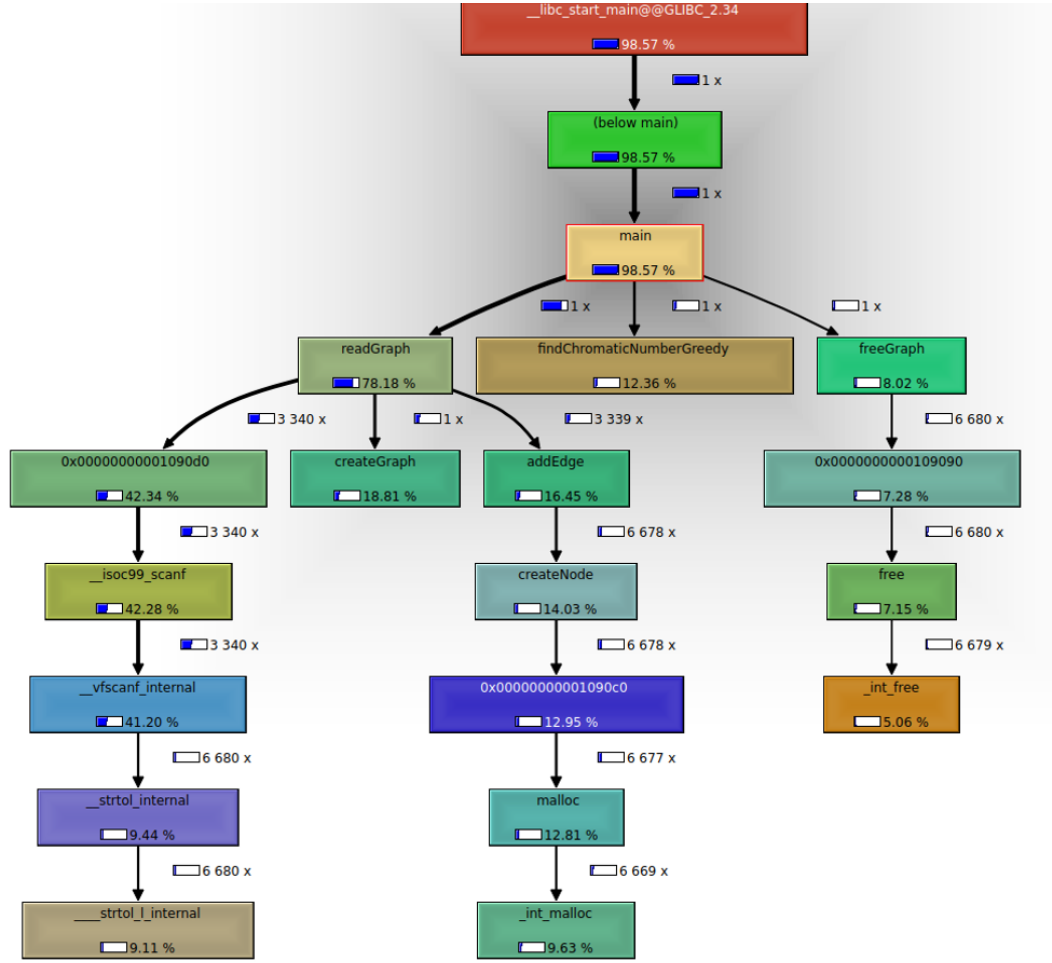


Fig. 11. CallGraph Analysis: Greedy

- **Backtracking:** The bottleneck is deep recursion (`graphColoringBacktrack`). Although the heap memory used is minimal (4 MB), execution time rivals that of the Greedy algorithm without scaling to larger graphs.



Fig. 12. CallGraph Analysis: Backtracking

5 Conclusions

- If speed is critical and the graphs are large, Welsh-Powell is the best choice. If a longer execution time is acceptable for potentially higher accuracy, DSatur is the preferred option, though its runtime cost increases dramatically.
- DSat provides the highest quality (fewest colors used), but it is extremely slow on large graphs.
- Welsh-Powell appears to offer the "best value": it is the fastest algorithm (even faster than Greedy) while delivering satisfactory solution quality (better than Greedy).
- For small-scale, low-complexity graphs (fundamental tests), the choice of algorithm has no practical impact on execution speed; all variants execute instantaneously.
- On special graph topologies, complex algorithms (DSat) exhibit unpredictable behavior (ranging from optimal performance to severe degradation), whereas simpler algorithms (Greedy and especially Welsh-Powell) deliver fast, consistent performance.
- On graphs with well-defined mathematical structures, algorithm selection is negligible (all heuristics perform well). On unstructured (random) graphs, complex algorithms (DSat) offer a distinct advantage in solution quality.
- There is no single "best" algorithm. When execution speed and processing large data volumes are prioritized, **Welsh-Powell** is the superior choice. If minimizing resource usage (color count) is paramount and processing time is secondary, **Standard DSatur** is the recommended solution.
- Across the evaluated graph set, memory analysis confirms the clear superiority of adjacency lists over adjacency matrices.
- For **real-time systems** (such as packet scheduling in routers, where decisions must be made within nanoseconds), Greedy is recommended.
- For **compiler register allocation** or **university timetabling** (where the algorithm is executed infrequently and optimal results are preferred over negligible runtime gains), DSatur or Welsh-Powell is recommended.

References

1. Briggs, P.: Register allocation via graph coloring. Rice University (1992)
2. Chow, F.C., Hennessy, J.L.: The priority-based coloring approach to register allocation. *ACM Transactions on Programming Languages and Systems (TOPLAS)* **12**(4), 501–536 (1990)
3. Dandashi, A., Al-Mouhamed, M.: Graph coloring for class scheduling. In: *ACS/IEEE International Conference on Computer Systems and Applications-AICCSA 2010*. pp. 1–4. IEEE (2010)
4. Galinier, P., Hertz, A.: A survey of local search methods for graph coloring. *Computers & Operations Research* **33**(9), 2547–2562 (2006)
5. Garey, M., Johnson, D., So, H.: An application of graph coloring to printed circuit testing. *IEEE Transactions on circuits and systems* **23**(10), 591–599 (1976)
6. Kraveva, R., Kravev, V., Katsarski, T.: An analysis between the welsh-powell and dsatur algorithms for coloring of sparse graphs. *International Journal of Electrical and Computer Engineering (IJECE)* **15**(4), 3867–3875 (2025)
7. Kubale, M.: Graph colorings, vol. 352. American Mathematical Soc. (2004)
8. Leighton, F.T.: A graph coloring algorithm for large scheduling problems. *Journal of research of the national bureau of standards* **84**(6), 489 (1979)
9. Lewis, R.: *A Guide to Graph Colouring: Algorithms and Applications*. Computer Science, Springer International Publishing (2015), <https://books.google.ro/books?id=RyfUCgAAQBAJ>
10. Olariu, S., Randall, J.: Welsh-powell opposition graphs. *Information Processing Letters* **31**(1), 43–46 (1989)
11. Thadani, S., Bagora, S., Sharma, A.: Applications of graph coloring in various fields. *Materials Today: Proceedings* **66**, 3498–3501 (2022). <https://doi.org/10.1016/j.matpr.2022.06.392>
12. Welsh, D.J.A., Powell, M.B.: An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal* **10**(1), 85–86 (01 1967). <https://doi.org/10.1093/comjnl/10.1.85>, <https://doi.org/10.1093/comjnl/10.1.85>
13. Zaini, D.D., Vincensius, H., Widjaja, K.A.N.U., Nurhasanah, Handoyo, A.T.: Implementing welsh-powell algorithm on coloring the map of west java. In: *Proceedings of the 8th International Conference on Sustainable Information Engineering and Technology*. pp. 679–684 (2023)